SECURITY ENGINEERING / RELEASE REVIEW

AssetTrust v0.46

Tenant row-level security - implementation report

RELEASE DECISION: NO-GO - do not deploy with customer data.

Executive summary

This checkpoint introduces PostgreSQL row-level security (RLS) for three **private PDF-layout administration tables**. The application now sets a validated, transaction-local tenant context before protected queries. Production startup refuses privileged database roles and missing RLS policies. This is a **staged rollout**, not complete database-wide isolation.

Scope of protection

Protected table	Enforcement
report_pdf_layout_proposals	FORCE RLS; organization read/write policy
report_pdf_layout_revisions	FORCE RLS; organization read/write policy
report_pdf_layout_active	FORCE RLS; organization read/write policy

Defensive design

The administrator path requires the current MFA-verified user before applying tenant scope. The recipient path derives its tenant from a verified, locked share record - never from recipient input. A missing tenant context denies access by policy. The context is set using a parameterized PostgreSQL call scoped to a transaction, not to the pooled connection.

Important limitation

Custom PostgreSQL tenant settings are not independent authorization credentials. Someone with unrestricted SQL execution or elevated database privileges could manipulate them. Parameterized queries, least-privilege database credentials, DB network isolation and application-level permission checks therefore remain essential.

TEST RESULTS AND EVIDENCE

Offline verification

Test area	Result
New v0.46 security tests	13 / 13 passed
Historical and current version suites (v03-v46)	44 / 44 commands passed
Offline commands incl. architecture, syntax and CI	47 / 47 commands passed
SQL migration source fingerprints	21 / 21 matched
TypeScript and TSX syntax-only scan	102 source files; 0 diagnostics
Current-source secret scan	1,726 files; 0 high-confidence matches

What these results do not prove

Passing offline tests and syntax transpilation does not prove that the application compiles, the database RLS rules function in live PostgreSQL, or real users are isolated. The v0.46 PostgreSQL contract exists and is guarded to refuse production databases, but it was **not executed** in this environment.

Outstanding release blockers

Real database proof. No disposable PostgreSQL server and installed pg package were available. Apply migration 021 and execute the two-tenant RLS contract on Kali.

Reproducible build. No genuine npm lockfile, no installed dependencies and no compiled application. Full build attempt failed on missing dependencies.

Tenant-wide protection. Users, assets, cases, documents, evidence, reports and other sensitive tables are outside this first RLS migration.

Production runtime privileges. A distinct non-owner, non-superuser, non-BYPASSRLS application account is not provisioned or verified on the actual deployment.

Security operations. Full Git history scan, key rotation, malware scanning, dependency audit, authenticated browsers, HTTPS/SSH configuration, and penetration tests remain unverified.

NEXT RELEASE GATES

Actions required on Kali Linux

1. Take a verified PostgreSQL backup, check restoration, and securely retain document encryption keys and report-signing keys.

2. Produce and review a genuine package-lock.json, then run npm ci, full build, npm test and dependency vulnerability checks.

3. Apply migration 021 using a migration-owner role to an explicitly disposable PostgreSQL instance; run the real v0.46 RLS contract.

4. Provision separate least-privileged production runtime credentials. Confirm the API refuses owner/superuser/BYPASSRLS roles and missing FORCE RLS.

5. Perform A/B organization tests, user role tests, MFA actions, recipient redemption and PDF export across separate API servers; check pool reuse after commits and rollbacks.

6. Extend RLS to remaining sensitive data domains without breaking legitimate public verification. Include foreign-key checks, policy reviews and a staged rollback plan.

7. Complete historical Git secret scans, key rotation, malware uploads, TLS/SSH/firewall review, backup restores and independent adversarial testing.

Release requirement

All critical/high security findings must be resolved and every mandatory control supported by live test evidence. This checkpoint makes progress on data isolation but does not approve any production launch. AssetTrust reports remain non-official evidence reports, not government land-title certificates.

Supporting files: V46_RLS_ARCHITECTURE.md, V46_KALI_VALIDATION.md, TEST_EVIDENCE_V46.md, V46_HANDOVER.md.